Orbit Propagation 2 – Numerical Methods for Orbit Propagation

Elizabeth Klioner M.Sc., Niklas Meyners M.Sc. | Chair of Space Technology | Institute of Aeronautics and Astronautics (ILR)

Introduction

- Numerical methods aim to find a numerical solution of an Ordinary Differential Equation (ODE) at intermediate values of an independent variable
- Results are valid only for initial parameter set
- Solution deteriorates over time, e.g. the further the distance from the initial parameter set, the lower the accuracy
- Numerical orbit propagation for special perturbation techniques is an Initial Value Problem (IVP)
  - Other applications are often IVPs, but Boundary Value Problems (BVPs) occur as well
- Focusing on methods to numerically solve IVPs in the following

Nomenclature (1)

- Systems of n ODEs for n unknown functions $x_1(t), \dots, x_n(t)$:

$$\dot{x}_1(t) = f_1\big(t, x_1(t), \dots, x_n(t)\big)$$
$$\vdots \quad = \quad \vdots$$
$$\dot{x}_n(t) = f_n\big(t, x_1(t), \dots, x_n(t)\big)$$

- Expressed in vector form:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}\big(t, \boldsymbol{x}(t)\big)$$

Nomenclature (2)

- Linear ODE systems can be expressed in vector-matrix form:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{A}(t)\boldsymbol{x}(t) + \boldsymbol{b}(t)$$

- Autonomous differential system: $\boldsymbol{f}$ does not explicitly depend on the independent variable

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}\big(\boldsymbol{x}(t)\big)$$

➔ When the independent variable is time, it is also called time-invariant system

- In the following (t) will be omitted as it is implied, e.g. $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$

Reduction of Order

- A scalar ODE of order n can be expressed as a n-dimensional system of first-order ODEs

$$u^{(n)} = G\left(t, u, \dot{u}, \dots, u^{(n-1)}\right)$$

➔ Set $\boldsymbol{x}$ so that: $\boldsymbol{x} = [x_1, x_2, \dots, x_n]^T \equiv \left[u, \dot{u}, \dots, u^{(n-1)}\right]^T$

- The system follows to:

$$\dot{x}_1 = x_2$$
$$\vdots \quad = \quad \vdots$$
$$\dot{x}_{n-1} = x_n$$
$$\dot{x}_n = G(t, x_1, \dots, x_n)$$

Converting Non-Autonomous ODE Systems

- A non-autonomous, n-dimensional first-order system can be rewritten as a $n+1$-dimensional system:

  ➔ Set $\boldsymbol{y}$ so that: $\boldsymbol{y} = [y_1, y_2, \dots, y_{n+1}]^T \equiv [t, x_1, \dots, x_n]^T$

- Add the ODE $\frac{d}{dx} t = 1$ to the system:

$$\dot{y}_1 = 1$$
$$\dot{y}_2 = f_1(y_1, \dots, y_{n+1})$$
$$\vdots \quad = \quad \vdots$$
$$y_{n+1} = f_n(y_1, \dots, y_{n+1})$$

Discretization For Numerical Methods (1)

- To numerically solve an IVP, the solution is approximated at discrete steps
- The interval of the IVP $[t, T]$ is discretized as $[t_i, t_{i+1}], i \in \mathbb{N}$ with the step size h:

$$h = t_{i+1} - t_i$$

➔ Equidistant step size

- Integral representation of the solution of the IVP:

$$\boldsymbol{x}(T) = \boldsymbol{x}(t) + \int_t^T \boldsymbol{f}\big(\tau, \boldsymbol{x}(\tau)\big) d\tau$$

- $\boldsymbol{f}$ is approximated with a Lagrange interpolation polynomial using j intermediate nodes at

$$t_{ij} = t_i + \varrho_j h, \qquad j = 1, \dots, m, \qquad m \in \mathbb{N}, \qquad 0 \leq \varrho_j \leq 1$$

Discretization For Numerical Methods (2)

- $\boldsymbol{x}_{ij}^{h}$ is an approximation of the exact solution $\boldsymbol{x}(t_{ij})$ at the node t_{ij}
  - $\boldsymbol{x}_{ij}^{h}$ is unknown after first discretization, second one is needed using the same grid
- The discretized solution follows to

$$\boldsymbol{x}_{i+1}^{h} = \boldsymbol{x}_{i}^{h} + h\sum_{j=1}^{m}\beta_j \boldsymbol{k}_j, \quad \boldsymbol{k}_j = \boldsymbol{f}\left(t_i + \varrho_j h, x_i^h + h\sum_{l=1}^{m}\gamma_{jl}\boldsymbol{k}_l\right), j = 1, \dots, m$$

General form of a m stage Runge-Kutta Method

General Form of Runge-Kutta Methods

$$\boldsymbol{x}_{i+1}^{h} = \boldsymbol{x}_{i}^{h} + h\sum_{j=1}^{m}\beta_j \boldsymbol{k}_j, \quad \boldsymbol{k}_j = \boldsymbol{f}\left(t_i + \varrho_j h, x_i^h + h\sum_{l=1}^{m}\gamma_{jl}\boldsymbol{k}_l\right), j = 1, \dots, m$$

General form of a m stage Runge-Kutta Method

- The term updating $\boldsymbol{x}_i^h$ contains the increment function $\boldsymbol{\Phi}$:

$$\boldsymbol{x}_{i+1}^{h} = \boldsymbol{x}_{i}^{h} + h\boldsymbol{\Phi}(t_i, \boldsymbol{x}_i, h)$$

- The superscript h denotes the dependence on the step size and is omitted in the following

Butcher Tableaus

- Coefficients ϱ, γ, β are often expressed in Butcher tableaus in compact form

$$\begin{array}{c|ccc} \varrho_1 & \gamma_{11} & \cdots & \gamma_{1m} \\ \vdots & \vdots & \ddots & \vdots \\ \varrho_m & \gamma_{m1} & \cdots & \gamma_{mm} \\ \hline & \beta_1 & \cdots & \beta_m \end{array}$$

- Expressed with matrix & vectors:

$$\begin{array}{c|c} \boldsymbol{\varrho} & \boldsymbol{\Gamma} \\ \hline & \boldsymbol{\beta}^T \end{array}$$

Euler Method

- Simplest method to numerically solve an IVP
- Solution $\boldsymbol{x_{i+1}}$ is approximated using the slope at t_i:

$$\boldsymbol{x_{i+1}} = \boldsymbol{x_i} + h\boldsymbol{f}(t_i, \boldsymbol{x_i})$$

- Accuracy is dependent on the step size h
- Butcher tableau of forward Euler method ➔

0	0
	1

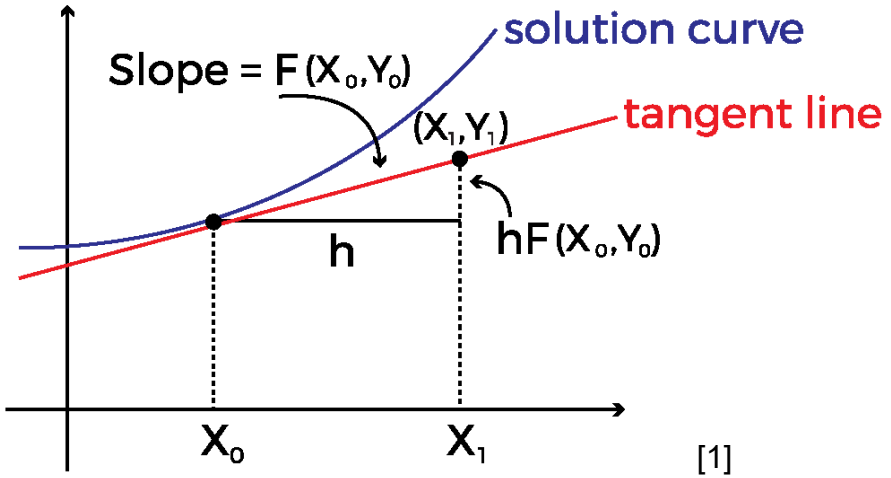
- Backward Euler method $\boldsymbol{x_{i+1}} = \boldsymbol{x_i} + h\boldsymbol{f}(t_{i+1}, \boldsymbol{x_{i+1}})$

  ➔ Slope is calculated at t_{i+1}, hence “backward”

- Butcher tableau of backward Euler method ➔

1	1
	1

- Both single step methods



[1]

Note: In figure different nomenclature is used
t_0 is X_0, x_0 is Y_0

Classification of Methods

Single-step methods

- Next step is calculated based only on previous step

  ➔ no “history” is being considered or saved

- Example: Runge-Kutta (RK) methods

Multi-step methods

- Next step is based on multiple steps before
- Higher orders of accuracy achievable

Runge-Kutta methods

- Combination of Taylor series and Euler method

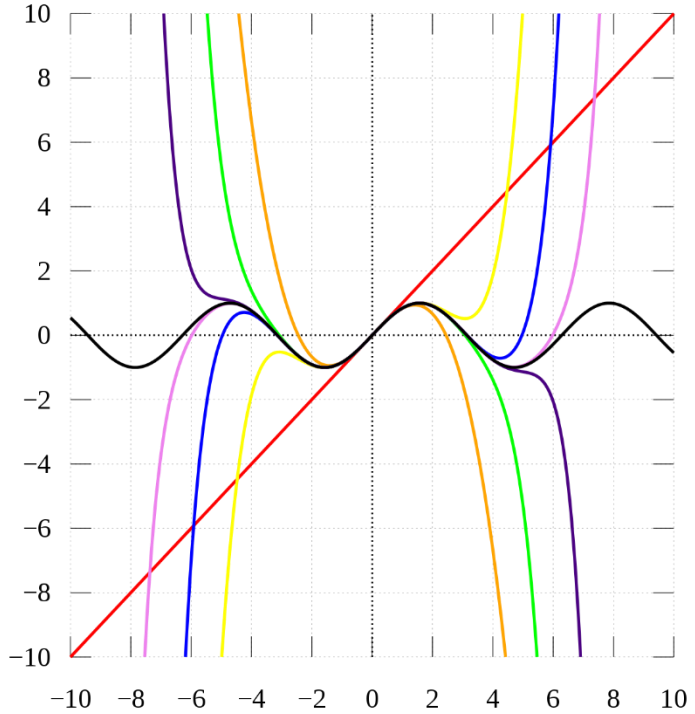
  ➔ replace high order derivates with some 1st order derivatives between t_i and $t_i + h$

  ➔ Integral is approximated using a weighted sum of evaluations of the 1st order derivative
- Runge-Kutta methods have an order p and a number of evaluations per stage m ($p \leq m$)
- Various versions (RK4, RK8, RKF, etc.)

Basis: Taylor Series

- Idea is to approximate a function using its derivates of higher order at a single point $x(t_0)$
- Allows to approximate a function at $x(t_0 + T)$ given $x(t_0)$

$$x(t_0 + T) = x(t_0) + T\dot{x}(t_0) + \frac{1}{2}T^2\ddot{x}(t_0) + \cdots = \sum_{k=0}^{\infty} \frac{1}{k!} T^k \left(\frac{d^k}{dt^k} x(t) \right) \Bigg|_{t=t_0}$$



- In general: the higher the order, the more accurate the approximation

[2]

Classical 4th order Runge-Kutta Method (1)

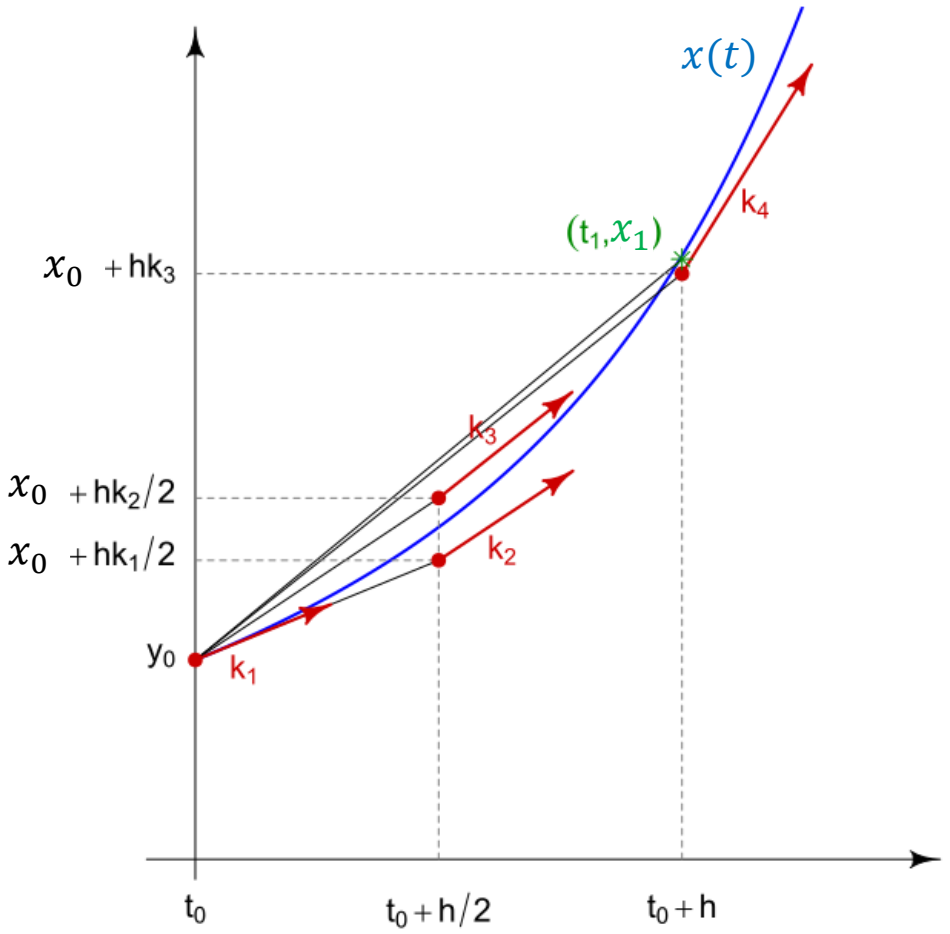
- Often called RK4

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \frac{1}{6}h(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4)$$

$$\boldsymbol{k}_1 = \boldsymbol{f}(t_i, \boldsymbol{x}_i), \qquad \boldsymbol{k}_2 = \boldsymbol{f}\left(t_i + \frac{1}{2}h, \boldsymbol{x}_i + \frac{1}{2}h\,\boldsymbol{k}_1\right)$$

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(t_i + \frac{1}{2}h, \boldsymbol{x}_i + \frac{1}{2}h\,\boldsymbol{k}_2\right), \qquad \boldsymbol{k}_4 = \boldsymbol{f}(t_i + h, x_i + h\,\boldsymbol{k}_3)$$

- $\boldsymbol{k}_1$ is calculated using the forward Euler method

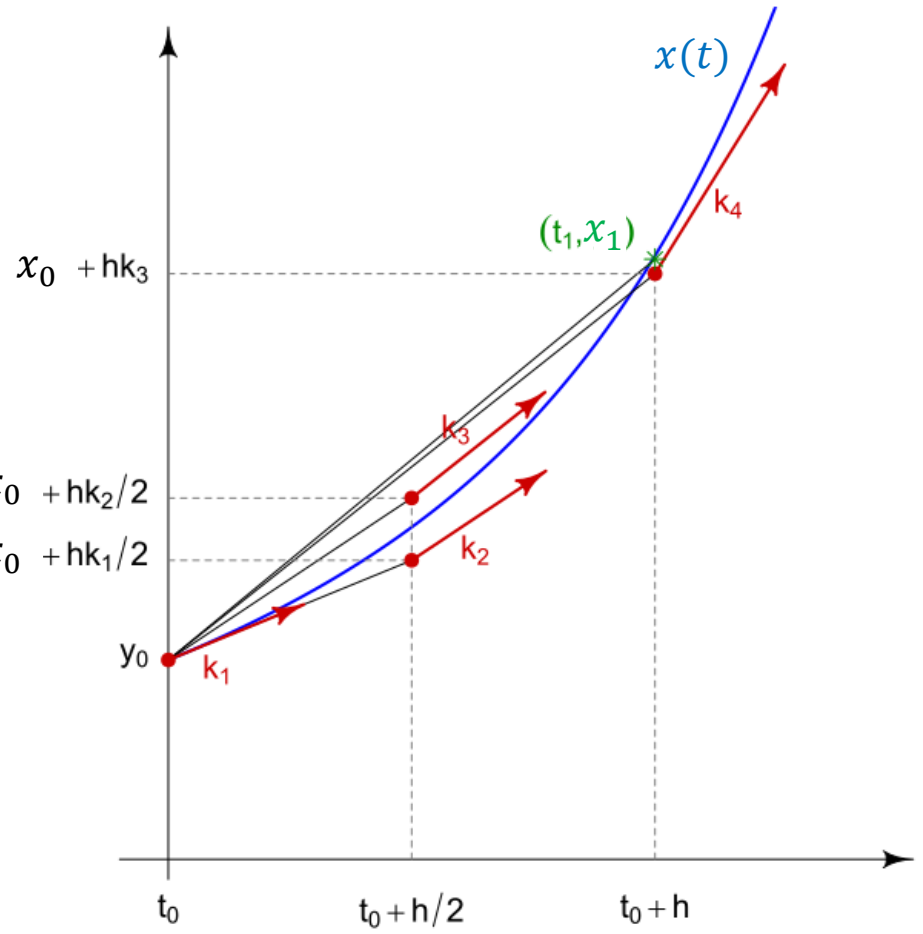


Modified, [3]

Classical 4th order Runge-Kutta Method (2)

- Butcher tableau →
- RK4 is an explicit method
  - $\gamma_{ij} = 0$ for i $\leq j$
  - Coefficient matrix Γ is a lower triangular matrix

0	0	0	0	0
$\frac{1}{2}$	$\frac{1}{2}$	0	0	0
$\frac{1}{2}$	0	$\frac{1}{2}$	0	0
1	0	0	1	0
	$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{6}$



Modified, [3]

- Local truncation error, i.e. error per one iteration, is on the order of $\mathcal{O}(h^5)$
- Global truncation error, i.e. accumulated error over all iterations, is on the order of $\mathcal{O}(h^4)$

Runge-Kutta-Fehlberg Methods

- Embedded methods, i.e. the coefficient matrix of lower-order RKF methods is a submatrix of higher-order RKF methods
- For RKF4:
  - $\boldsymbol{\Gamma} \in \mathbb{R}^{5\times5},\ \boldsymbol{\varrho}, \boldsymbol{\beta} \in \mathbb{R}^{5\times1}$
- For RKF5:
  - $\boldsymbol{\Gamma} \in \mathbb{R}^{6\times6},\ \boldsymbol{\varrho}, \boldsymbol{\beta} \in \mathbb{R}^{6\times1}$
- Only $\boldsymbol{\beta}$ is completely different!
- Embedded methods allow simple-to-implement variable step size control

0	0	0	0	0	0	0	
$\frac{1}{4}$	$\frac{1}{4}$	0	0	0	0	0	
$\frac{3}{8}$	$\frac{3}{32}$	$\frac{9}{32}$	0	0	0	0	zusätzliche Spalte bei RKF5
$\frac{12}{13}$	$\frac{1932}{2197}$	$-\frac{7200}{2197}$	$\frac{7296}{2197}$	0	0	0	
1	$\frac{439}{216}$	-8	$\frac{3680}{513}$	$-\frac{845}{4104}$	0	0	
$\frac{1}{2}$	$-\frac{8}{27}$	2	$-\frac{3544}{2565}$	$\frac{1859}{4104}$	$-\frac{11}{40}$	0	zusätzliche Zeile bei RKF5
	$\frac{25}{216}$	0	$\frac{1408}{2565}$	$\frac{2197}{4104}$	$-\frac{1}{5}$		β_i für RKF4
	$\frac{16}{135}$	0	$\frac{6656}{12825}$	$\frac{28561}{56430}$	$-\frac{9}{50}$	$\frac{2}{55}$	β_i für RKF5

[4]

Variable Step Size and Step Size Control

- Step size is not fixed anymore and is dependent on the step i: $h_i = t_{i+1} - t_i$
- Basic idea: Estimate error of the lower-order approximation using the higher-order method
  - For RKF: Approximate the 4th order method's error using the 5th order method

    → RKF4(5)
- Either local truncation error (Error Per Step, EPS) or global truncation error (Error Per Unit Step, EPUS) is estimated
  - For RKF4: EPS is on the order of $\mathcal{O}(h^5)$, EPUS is on the order of $\mathcal{O}(h^4)$
- Showing EPS-based step size control

Step Size Control with EPS (1)

- EPS-based step size control: h_i has to be set so that:

$$|h_i \boldsymbol{\delta}(t_{i+1}, \boldsymbol{x}_i(t_{i+1}); h_i)| \approx \epsilon_{abs}$$

With the (user-defined) absolute tolerance ϵ_{abs}

- The error estimation e_{i+1} for $|h_i \boldsymbol{\delta}(t_{i+1}, \boldsymbol{x}_i(t_{i+1}); h_i)|$ is

$$e_{i+1} = \frac{|\hat{\boldsymbol{x}}_{i+1} - \boldsymbol{x}_{i+1}|}{h}$$

- $\hat{\boldsymbol{x}}_{i+1}$: approximation of the higher-order method on the order of q
- $\boldsymbol{x}_{i+1}$: approximation of the lower-order method on the order of p

Step Size Control with EPS (2)

- Constant α is computed with a safety factor of 0.9:

$$\alpha = 0.9 \left(\frac{\epsilon_{abs}}{e_{i+1}} \right)^{\frac{1}{p+1}}$$

- **Case 1**: if $\frac{e_{i+1}}{\epsilon_{abs}} \leq 1$, the step size h_i for the current step is accepted and the step size $h_{i+1}$of the subsequent step is increased with $h_{i+1} = \alpha\, h_i$
- **Case 2**: if $\frac{e_{i+1}}{\epsilon_{abs}} > 1$, the step size h_i for the current step is not accepted. A new step size $h_{i,n}$ is computed by decreasing h_i with $h_{i,n} = \alpha\, h_i$ and the step is repeated

Step Size Control with EPS (3)

- Step size increase for h_{i+1} and decrease for $h_{i,n}$ is based on e_{i+1} being on the order of $p+1$

  → h (for new step size h_{i+1} or $h_{i,n}$) is computed so that:

$$\left(\frac{h}{h_i}\right)^{p+1} \approx \frac{\epsilon_{abs}}{e_{i+1}}$$

- If $e_{i+1} \approx 0$, α cannot be computed → In case 1 h_{i+1} is set to a user-defined h_{max}
  - It is advisable to set a h_{min} to prohibit roundoff errors

REPS-Based Step Control

- Step size can also be controlled using the relative local truncation error REPS (Relative Error Per Step)
  - Approach is equivalent to EPS, but setting h_i so that:

$$\frac{|h_i \boldsymbol{\delta}(t_{i+1}, \boldsymbol{x}_i(t_{i+1}); h_i)|}{|\boldsymbol{x}_{i+1}|} \approx \epsilon_{rel}$$

  - ϵ_{rel} is the user-defined relative tolerance

Combined Approach (1)

- Step size control is often implemented using a combined approach
- Both EPS and REPS are used to set the step size in an effective way
- h_i is to be set so that:

$$e_{i+1} \approx \epsilon_{abs} + \epsilon_{rel} \max(|\boldsymbol{x}_i|, |\boldsymbol{x}_{i+1}|)$$

→ For embedded methods, the unnormalized error can be estimated with:

$$e_{i+1} \approx \boldsymbol{x}_{i+1} - \hat{\boldsymbol{x}}_{i+1} = h\left(\boldsymbol{\Phi} - \hat{\boldsymbol{\Phi}}\right)$$

Combined Approach (2)

- The tolerances are combined to get ϵ:

$$\epsilon = \epsilon_{abs} + \epsilon_{rel} \max(|\boldsymbol{x}_i|, |\boldsymbol{x}_{i+1}|)$$

- With ϵ, the estimated error e_{i+1} is normalized:

$$e_{norm} = \left| \frac{e_{i+1}}{\epsilon} \right|$$

- α is computed with:

$$\alpha = 0.9 \left(\frac{1}{e_{norm}} \right)^{\frac{1}{p+1}}$$

Combined Approach (3)

- **Case 1**: if $e_{norm} \leq 1$, the step size h_i for the current step is accepted and the step size h_{i+1} of the subsequent step is increased with $h_{i+1} = \alpha\, h_i$
- **Case 2**: if $e_{norm} > 1$, the step size h_i for the current step is not accepted. A new step size $h_{i,n}$ is computed by decreasing h_i with $h_{i,n} = \alpha\, h_i$ and the step is repeated

Remarks on Step Size Control (1)

- Step size control allows for efficient computation as step size is only increased when tolerances are not met
- Step size control is almost always implemented for embedded RK methods as they share function evaluations

Remarks on Step Size Control (2)

- 3-body simulation with RKF with step size control
- GIF shows computational time

  → More time is spent when bodies are closer



[5]

Computational Effort of RK Methods

- Computational effort depends mainly on the number of function evaluations per step m

  → For trajectory propagation one function evaluation can be quite demanding depending on the accuracy of the model

- Order p and number of evaluations per step m differ greatly the higher p is

p	1	2	3	4	5	6	7	8	9	10	12	14
m_p	1	2	3	4	6	7	9	11	15	17	25	35

[4]

- Not beneficial to use very high-order methods if the accuracy is not required

Other Runge-Kutta Methods (1)

Runge-Kutta-Dormand-Prince

- Commonly used family of embedded methods
  - MATLAB solver ode45 is implemented using Runge-Kutta-Dormand-Prince

Runge-Kutta-Nyström

- Used for second-order differential equations

$$\ddot{x} = f(t, r, \dot{x})$$

Other Runge-Kutta Methods (2)

Runge-Kutta-Nyström-Fehlberg

- Combination of RKN and RKF
  - Allows for step size control and second-order differential equations

Implicit Runge-Kutta methods

- $\boldsymbol{\Gamma}$ is no lower triangular matrix
- Coefficients $\boldsymbol{\Gamma}, \boldsymbol{\varrho}, \boldsymbol{\beta}$ are given by system of equations which need to be solved

Some general remarks on single-step methods

- The lower the step size, the higher accuracy

  → even low-order methods can produce high accuracy but(!) with high computational time
- Step size control is always preferable

  → if step size is fixed, a bad initial step size guess can cause severe errors
- Step size control allows to pre-determine the desired accuracy
- Single-step methods are especially useful for rapidly changing functions

  → step size can be changed in each step

Multi-step methods

- Idea: use not only previous step but several previous steps
- Integral is approximated using Newton polynomials

General:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(t, \boldsymbol{x}) \;\rightarrow\; \boldsymbol{x}(t_{i+1}) = \boldsymbol{x}(t_i) + \int_{t_i}^{t_i+h} \boldsymbol{f}\big(t, \boldsymbol{x}(t)\big) dt$$

→ An approximation can be achieved with $\boldsymbol{f}(t, \boldsymbol{x}_i) \approx \boldsymbol{f}(t_0 + h, \boldsymbol{x}_0 + nh)$

→ The relation can be in general expressed as

$$\sum_{j=0}^{k} \alpha_j x_{n+j} = h \sum_{j=0}^{k} \beta_j f_{n+j}$$

→ x_{n+k} shall be predicted using $x_n, \ldots, x_{n+k-1}$

Multi-step method

Predictor $\beta_k = 0$

- Explicit method that extrapolates the next step based on the past ones (“predictor”)

$$x_{n+k} = h\sum_{j=0}^{k} \beta_j f_{n+j} - \sum_{j=0}^{k} \alpha_j x_{n+j}$$

Corrector $\beta_k \neq 0$

- Implicit method as some iterative method required to solve for f_{n+k}

$$x_{n+k} = h\beta_k f_{n+k} + h\sum_{j=0}^{k} \beta_j f_{n+j} - \sum_{j=0}^{k} \alpha_j x_{n+j}$$

Adams-Bashforth-Moulton

- Multi-step method, fixed step size
- Combination of a predictor and a corrector

Step 1 “prediction”: use predictor for an initial estimate of $\hat{x}_{n+k}$: $\hat{x}_{n+k}^{P}$

Step 2 “evaluation”: find the corresponding function value f_{n+k}^{P}

Step 3 “corrector”: use f_{n+k}^{P} to correct the initial guess of $\hat{x}_{n+k}^{P}$ to obtain an improved $\hat{x}_{n+k}$

Step 4 “final”: find $f_{n+k} = f(t_{n+k}, \hat{x}_{n+k})$

Some general remarks on multi-step methods

- Methods cannot initialize themselves

  → need another method to approximate the first steps (e.g. RK)
- multi-step method reduces total number of function evaluations

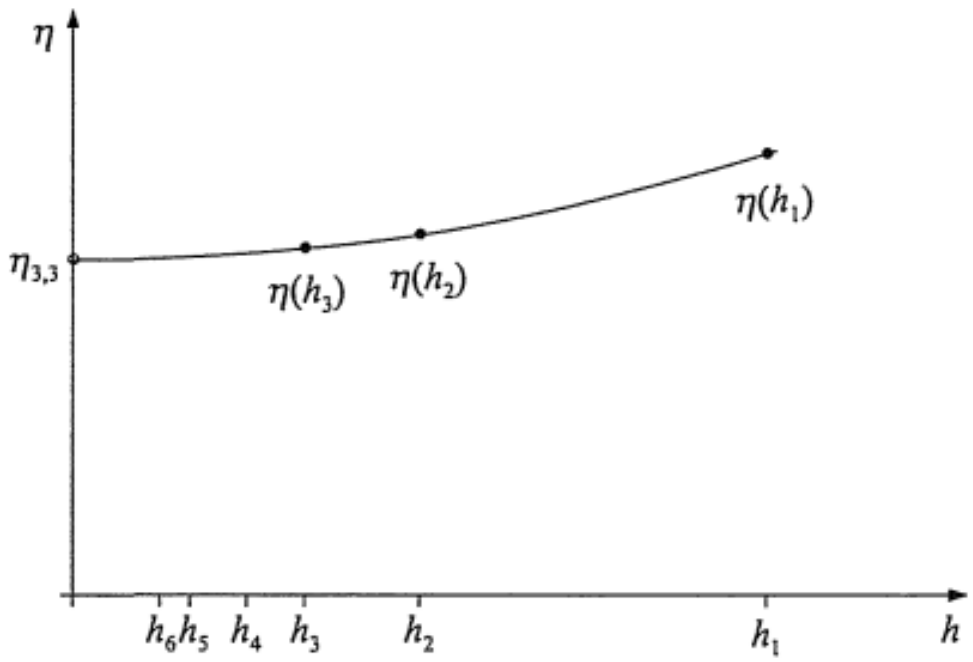
  → Useful for complicated functions with lots of arithmetic operations
- Multi-step can also have variable step sizes and variable order

  → variable order does not need initial guess as it can start at 1st order

Extrapolation methods

- Idea: integrate over the same interval for various step sizes h using one integration technique

  → extrapolate the result for $h \to 0$
- Examples: DIFSYS1/DIFSYS2 or ODEX1/ODEX2
- Good method for very various applications



[6]

Some remarks on accuracy and its evaluation

- Numerical method must be chosen specifically for the problem at hand

  → and so must the testing algorithm
- Integration methods only provide the solution at the beginning and end of the integration interval

  → in-between interpolation necessary (causes again errors)

  → continuous methods can be useful (not always suitable)
- Example evaluation for conservative forces: Use method backwards to check whether you get initial values

Information and Tips for Assignment 1

- Mind the units for $\dot{\Omega}$ when modelling the J2 perturbation for the Kepler equation
  - Multiple formulas can be found with units like per day, per second, per orbit
- Spherical coordinates are sufficiently precise for the ground track
  - You can still use geodetic coordinates, if preferred.
- ICRF2ITRF transformation → approximate using Θ only
- Units: end result km, adaptable if order of magnitude is too small
  - In general: “reasonably” precise, most important is consistency

J2 Perturbation Accelerations

$$a_x = -\frac{3J_2\mu R_E^2\, r_x}{2r^5}\left(1 - \frac{5r_z^2}{r^2}\right)$$

$$a_y = -\frac{3J_2\mu R_E^2\, r_y}{2r^5}\left(1 - \frac{5r_z^2}{r^2}\right)$$

$$a_z = -\frac{3J_2\mu R_E^2\, r_z}{2r^5}\left(3 - \frac{5r_z^2}{r^2}\right)$$

- Here, $\boldsymbol{r}$ is the position vector in the Earth-fixed frame
- International Terrestrial Reference Frame is used as frame

Image index

[1] https://calcworkshop.com/first-order-differential-equations/eulers-method-table/

[2] https://en.wikipedia.org/wiki/Taylor_series#/media/File:Sintay_SVG.svg

[3] https://commons.wikimedia.org/wiki/File:Runge-Kutta_slopes.svg, modified

[4] Hermann, Martin. „Numerik gewöhnlicher Differentialgleichungen Band 1: Anfangswertprobleme und lineare Randwertprobleme“, 2. ed., De Gruyter, 2017

[5] https://commons.wikimedia.org/wiki/File:Periodic_3-body_RKF_Integration.gif

[6] Montenbruck, Oliver, Eberhard Gill, and Fh Lutze. "Satellite orbits: models, methods, and applications." *Appl. Mech. Rev.* 55.2 (2002): B27-B28.